

qala

Solution Factory Operating System

Solution System Design Document · Version 3.0

Solution Domain Architecture & Specification
March 2026

CONFIDENTIAL	INTERNAL USE ONLY	DRAFT v3.0
--------------	-------------------	------------

1. Executive Summary

qala is a Solution Factory Operating System (SFOS) — a unified, hierarchical platform for designing, building, managing, and operating solutions of every type, at any scale. The Solution System is the primary domain of the platform. The Solution is the central root element around which every subsystem, environment, factory, workflow, and governance instrument is organized.

Every deliverable managed in qala — whether a software application, a physical good, a market-facing product, a delivered service, or a complex platform — is represented as a typed Solution with a consistent structure, governed lifecycle, and traceable value chain. From a solo developer organizing a personal project to a global enterprise managing thousands of products and services, qala provides a single standardized interface and operating model.

qala is itself a solution factory — the root factory — capable of producing tiered, hierarchical solution factories, development environments, and solution models. The platform treats every deliverable as a typed Solution with a consistent structure, governed lifecycle, and traceable value chain.

Core Principle: Every solution, regardless of type or scale, is created, maintained, and operated within a standardized, reproducible, and governed environment. The Solution is the root element of the qala model — everything else exists to enable the Solution.

1.1 Platform Objectives

qala is designed to achieve the following primary outcomes:

- Standardize the creation and management of all solution types — applications, systems, goods, products, services, and platforms — under a single operating model
- Enable reproducible, hermetically sealed development and build environments at every tier of the factory hierarchy
- Provide end-to-end lifecycle management from ideation through deployment, operation, and archival
- Integrate CI/CD, configuration management, security, testing, benchmarking, and observability into a coherent, automated system
- Scale seamlessly from individual developers to enterprise organizations without changing the underlying model
- Embed AI-driven intelligence throughout — in optimization, prediction, anomaly detection, and recommendation
- Establish a governed, traceable value chain for every solution from concept to operational deployment
- Support sustainability and reusability principles: closed-loop solution design, resource recycling, and eco-aware lifecycle management

1.2 Scope

This document covers the full architectural design of the qala platform including the Solution Model, Solution Development Environment, Solution Factory, Hermetic Build System, CI/CD and Configuration Management, Testing and Benchmarking, Security and Privacy, Release and

Deployment, Governance and Compliance, Data Platform, AI Integration, and complete API and microservice architecture.

1.3 Platform Ecosystem: kogi · ume · qala

qala operates as the output management layer of a broader three-platform ecosystem designed for systematic, deterministic outcomes:

Platform	Domain	Function	Relationship
kogi	Input Management	Manages inputs — requirements, signals, resources, and raw data entering the system	Upstream feeder to ume and qala
ume	Transformation Management	Manages transformations — processing, enriching, and refining inputs into structured work	Transforms kogi inputs into qala-ready work
qala	Output Management	Manages outputs — solutions, artifacts, and deployments produced from transformed inputs	Root platform; produces and governs all solutions

Together, kogi + ume + qala manage outcomes in a systematic and deterministic way. This document covers the qala platform exclusively.

2. Solution System Overview

The Solution System is the top-level domain of the qala platform. It defines the ontology for what a Solution is, how it is structured, how it is classified, and how it relates to every other system in the platform. All factories, environments, tooling, pipelines, and governance frameworks exist to serve the Solution System.

2.1 The Solution as Root Element

The Solution is the central, root element of the qala platform. Every object managed in qala either is a Solution, belongs to a Solution, or exists to support a Solution. The platform's information architecture radiates outward from this root.

Domain	Relationship to Solution	Description
Solution	Root Element	The fundamental unit of value; typed, versioned, composed of components, governed by lifecycle
Solution Component	Part-of Solution	A constituent element of a Solution; carries design, parts, materials, and IDs
Solution Part	Sub-element of Component	The lowest-level constituent: uniquely identifiable, sourceable, and reproducible
Solution Model	Blueprint of Solution	The design, blueprint, and prototype specification that defines a Solution before and during construction
Solution Testbed	Validates Solution	The environment and test suites used to verify a Solution against its acceptance criteria
Solution Factory	Produces Solution	The coordinated SDE network that manufactures Solutions in a consistent, repeatable way
Solution Vendor	Supplies to Solution	External or internal providers of parts, components, services, or capabilities
Solution Orchestration	Executes for Solution	Workflows and tasks that drive Solution creation and operation
Solution Data	Describes Solution	Metadata, features, versioning, and maturity classification
Solution Tooling	Enables Solution	Toolkits, toolsets, toolchains, and tools used to build Solutions
Solution Book	Documents Solution	Charter, notes, parts, vendors, binders, schedules, WBSs, playbooks, and all reference content
Solution Package	Distributes Solution	Bundled, versioned, distributable form of a Solution and its artifacts

Domain	Relationship to Solution	Description
Solution Registry	Catalogs Solutions	Authoritative platform-wide catalog of all Solutions, Models, Vendors, Tools, and Factories
Solution Portfolio	Aggregates Solutions	A governed collection of Solutions managed under a common Factory or organizational unit
Solution Chain / Set / Kit	Composes Solutions	Higher-order groupings linking multiple Solutions into coherent offerings or value chains

2.2 Solution Types

qala recognizes six canonical Solution types. Every artifact managed in the platform is classified as one of these types. The type determines default governance rules, testbed configurations, deployment patterns, and value chain instrumentation.

Solution Type	Description	Typical Outputs
Application	A software application with defined processes and user interactions	Executable, container image, API endpoint, web app
System	A composition of applications with coordinated behavior and shared infrastructure	Deployed system topology, integration contracts, shared services
Good	A tangible or digital deliverable produced as an output of a process	Physical component, digital file, media asset, design artifact
Product	A market-facing offering combining software, services, and/or goods	Versioned product release, SKU, distribution package
Service	A capability or function delivered to a consumer on-demand or continuously	Running service instance, SLA-governed capability, subscription offering
Platform	A substrate on which other Solutions are built, deployed, and operated	Infrastructure, runtime environment, developer platform, marketplace

3. Solution

A Solution is the fundamental unit of value in qala. It represents a realized answer to a problem, a goal, or an objective. Solutions are typed, versioned, composed of components, governed by lifecycle states, and linked to the factories, environments, and value chains that produced them.

3.1 Solution Metadata

Every Solution carries a canonical metadata record that persists in the Solution Registry and is versioned alongside the Solution itself.

Metadata Field	Type	Description
Unique ID	UUID v4	Globally unique identifier assigned at Solution creation; immutable
Name	string	Human-readable Solution name; unique within a Factory namespace
Version	semver	Semantic version string (MAJOR.MINOR.PATCH); managed by version control
Maturity	enum	Lifecycle maturity level: SANDBOX DEV NIGHTLY TEST CM
Solution Type	enum	Application System Good Product Service Platform
Owner	ref	User or organization responsible for the Solution's quality and lifecycle
Contributing Owners	ref[]	Additional owners with defined shared responsibilities
SDE Reference	ref	Pointer to the SDE in which this Solution is actively developed
Factory Reference	ref	Pointer to the Solution Factory that produced this Solution
Value Proposition	text	Concise statement of the problem this Solution solves and the value it delivers
Created At	datetime	ISO 8601 timestamp of Solution record creation
Updated At	datetime	ISO 8601 timestamp of last modification
Tags	string[]	Freeform labels for search, discovery, and classification
Status	enum	Active Deprecated Retired Archived
License	string	Applicable license governing use and distribution

3.2 Solution Maturity Model

Each Solution progresses through a defined maturity lifecycle. Promotion between stages requires passing configured gate criteria. Maturity governs which operations are permitted — for example, only CM-maturity Solutions may produce immutable release artifacts.

Stage	Code	Description	Gate Criteria
Sandbox	SANDBOX	Exploratory, unconstrained. No stability guarantees. Open creation.	None — open creation
Development	DEV	Active feature development. Nightly builds may break.	Solution record created; SDE provisioned; owner assigned; basic build success
Nightly	NIGHTLY	Automated nightly build and test cycle. Stability improving.	CI pipeline configured; build succeeds; unit tests passing; CI green on main
Test	TEST	Feature-complete. Full test suites running. Pre-release hardening.	All unit and integration tests passing; SAST clean; no P1 defects open
Control Managed	CM	Release-qualified. Under Configuration Management. Immutable.	All test suites passing; DAST clean; benchmarks met; CM board approval; build attestation generated

3.3 Solution Features List

A Solution's capabilities are described by a structured features list. Each feature carries a name, a brief summary, and a full feature specification that serves as the acceptance criteria definition.

Feature Field	Type	Description
Name	string	Short identifier for the feature
Brief	string	One-sentence summary of what the feature does and the value it delivers
Feature	text	Full specification: behavior, acceptance criteria, constraints, and related components
Priority	enum	Must-Have Should-Have Could-Have Won't-Have (MoSCoW)
Status	enum	Planned In Progress Completed Deprecated

Feature Field	Type	Description
Version Introduced	semver	The Solution version in which this feature was first delivered
Component Reference	ref[]	Links to Solution Components that implement this feature

3.4 Solution Value Proposition

Every Solution in qala maintains a structured value proposition record — a concise, verifiable statement of what problem the Solution solves, for whom, and what measurable outcomes it delivers. The value proposition is authored at the SANDBOX stage and refined at each maturity promotion.

- Problem Statement: the specific problem or gap the Solution addresses
- Target Audience: the primary consumer, user, or beneficiary of the Solution
- Differentiator: what makes this Solution preferable to alternatives
- Success Metrics: measurable indicators of Solution success at the CM stage
- Strategic Alignment: linkage to organizational goals, roadmap, or portfolio priorities

4. Solution Component & Solution Part

A Solution is decomposed into Components and then into Parts. This two-level decomposition model is type-agnostic — it applies uniformly to software applications, physical goods, services, and platforms. The same schema applies universally, enabling consistent tooling, governance, and analysis across all Solution types.

4.1 Component Structure

A Solution Component is the first-level decomposition of a Solution. Components carry design, manufacturing, and material information and can be assembled from lower-level Parts.

Component Field	Type	Description
Component ID	UUID	Unique identifier for this component within the Solution
Component Name	string	Human-readable name
Component Type	enum	module sub-assembly feature capability service interface process
Design / Blueprint	ref	Reference to the design specification or blueprint document for this component
Version	semver	Component-level version string; independently versioned from the Solution
Owner	ref	Person or team responsible for this component's quality and lifecycle
Solution Reference	ref	Foreign key back to the parent Solution
Parts List	ref[]	Ordered collection of Solution Parts belonging to this component
Interface Contracts	ref[]	Inbound and outbound interface definitions for this component
Test Suite Reference	ref	Link to the testbed suite validating this component
Maturity	enum	Component-level maturity, must not exceed parent Solution maturity

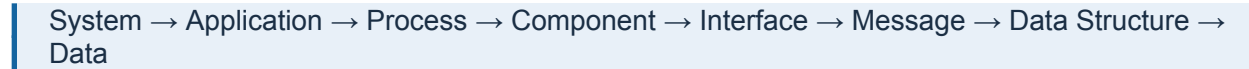
4.2 Solution Part

A Part is the lowest-level constituent of a Component. Parts carry manufacturing and material identity sufficient to uniquely identify, source, and reproduce them. A Part may represent a software library, a physical sub-component, a licensed service, or any other atomic supply item.

Part Field	Type	Description
Part ID	UUID	Unique identifier (platform-assigned UUID)
Part Number	string	External or internal part number (e.g., manufacturer PN, package name, library identifier)
Part Name	string	Human-readable name for the Part
Part Vendor	ref	The supplier, manufacturer, or open-source project that is the source of this Part
Part Material	string	Material classification: ABS plastic steel open-source library SaaS API compiled binary etc.
Part Design / Blueprint	ref	Reference to the drawing, specification, schema, or datasheet defining this Part
Part Version	semver	Version of this Part's design or supply revision; pinned in the dependency manifest
Verification Hash	string	Cryptographic hash (SHA-256) of the Part binary or specification for supply-chain verification
Part Tags	string[]	Freeform labels for categorization and search
License	string	License governing use and redistribution of this Part

4.3 The Universal Solution Structure Model

qala defines a universal containment hierarchy for decomposing any Solution. This hierarchy is consistent across all Solution types — the same structural grammar applies to a software application, a physical product, a delivered service, and a business platform.



Layer	Contains	Interface / Message Type	Notes
System	Applications	—	Top-level organizational boundary; contains one or more Applications
Application	Processes	—	Executable units with defined behavior and ownership
Process	Components	—	Logical groupings of functionality with defined scope

Layer	Contains	Interface / Message Type	Notes
Component	Interfaces	—	Implementation units with explicit input/output contracts
Interface	Messages + Imports/Exports	Inbound / Outbound	Boundary layer defining data contracts; explicit import and export declarations
Message	Data Structures	Event (dynamic) / State (static)	Typed message payload; Event messages are dynamic and time-bound; State messages are static snapshots
Data Structure	Data fields	—	Typed fields using the qala canonical data type system
Data	Primitive values	—	Leaf-level values using canonical types: bool, string, int, float, etc.

4.4 Data Type System

qala defines a canonical set of data types used across all Solution data structures. Custom types may be composed from these primitives and registered in the Solution Registry.

Category	Types
Primitive	bool, string, char, varchar
Numeric	int, float, double, null
Temporal	date, datetime, timestamp, duration
Collection	array, tuple, set, map
Reference	pointer, object
Special	custom (composed from primitives and registered in the type registry)

5. Solution Model

The Solution Model defines the formal specification for a Solution before and during its construction. It is the intellectual blueprint layer — containing designs, blueprints, architectures, mockups, and prototypes — that drives the manufacturing process executed by the Solution Factory. Every Solution has exactly one associated Solution Model; the Model is versioned alongside the Solution.

5.1 Model Elements

Model Element	Description	Artifact Types
Blueprint	Formal structural specification: architecture, interfaces, data contracts, and dependency graph	Architecture diagram, interface definition language (IDL), dependency manifest
Design	Visual and functional design artifacts defining appearance and behavior	Wireframes, schematics, drawings, design system references, UI specifications
Architecture	System-level structure: service topology, data flows, integration patterns, and deployment targets	System diagrams, ADRs (Architecture Decision Records), topology maps
Mockup	Static visual representation of interfaces and user-facing surfaces for review and sign-off	Screen mockups, physical renders, UI prototypes
Prototype	A working instantiation of a subset of Solution capabilities for validation and feedback	POC implementations, functional demos, simulation environments

5.2 Solution Playbooks, Plans & Roadmaps

Every Solution in qala may have associated planning and operational artifacts that capture its design intent, lifecycle plans, and operational procedures. These are managed within the Solution Book and linked from the Solution Model.

- Playbooks: step-by-step operational procedures for deployment, incident response, scaling, and onboarding
- Blueprints: formal architectural designs and component relationship diagrams
- Roadmaps: multi-horizon feature and capability plans with strategic alignment statements
- Timelines: chronological representations of Solution history and planned milestones
- Gantt Charts: schedule representations with dependencies, critical paths, and resource assignments
- Work Packages: bounded units of work with defined scope, owner, deliverable, and effort estimate

- Work Breakdown Structures (WBS): hierarchical decomposition of all work required to deliver the Solution

5.3 Solution Registry — Models Catalog

All Solution Models are registered in the Solution Models Registry, a sub-catalog of the platform-wide Solution Registry. The Models Registry provides lineage tracking, cross-reference services, version history, and governance assignment for every Blueprint, Design, Architecture, Mockup, and Prototype in the system.

6. Solution Testbed

The Solution Testbed is the verification and validation environment for a Solution. It defines the full battery of tests, test suites, testbed infrastructure, and quality gates that a Solution must pass before advancing through the maturity lifecycle. Testing in qala is a continuous, integrated activity — not a discrete phase.

6.1 Testbed Architecture

Testbed Element	Description
Test Case	The atomic unit: a single scenario with defined inputs, expected outputs, pass/fail criteria, and traceability to a requirement or feature
Test Suite	A named, ordered collection of related test cases targeting a specific feature, component, or quality concern
Test Plan	The master document defining scope, schedule, environments, resource assignments, and acceptance criteria for a release
Test Environment	An isolated, automatically provisioned environment matching the target deployment topology; torn down after execution
Test Run	A single execution of a test suite against a specific Solution build, producing a dated, versioned results record
Defect	A recorded failure: linked to the test case, the build, the commit, the release, and the owner responsible for resolution
Test Analytics	Aggregated metrics across runs: pass rates, trend analysis, defect density, coverage maps, MTTR, and AI-driven predictions

6.2 Test Types

Test Type	Scope	Automation Level	Pipeline Stage
Unit Tests	Individual functions, classes, and modules	Fully automated — every commit	CI build stage
Integration Tests	Service-to-service and module boundaries	Fully automated — every build	CI post-build stage
System Tests	End-to-end Solution behavior	Automated — nightly and release builds	CD staging stage
Performance / Benchmark	Throughput, latency, scalability under load	Automated — release candidates	CD performance gate

Test Type	Scope	Automation Level	Pipeline Stage
Security / SAST	Static vulnerability scanning of source and dependencies	Automated — every commit and build	CI security stage
Security / DAST	Dynamic vulnerability scanning of running application	Automated — builds targeting TEST+	CD staging security gate
Security / IAST	Instrumented testing during execution	Automated — integration test stage	CD integration stage
Security / SCA	Dependency and supply-chain vulnerability scanning	Automated — dependency resolution stage	CI dependency stage
Acceptance Tests (UAT)	Business requirements validation	Semi-automated — human sign-off required	CD acceptance gate
Prototype Tests	POC feasibility and design validation	Manual + AI-assisted feedback analysis	SANDBOX/DEV stages
Regression Tests	Prevention of previously resolved defects	Fully automated — every build	CI and CD stages
Penetration Tests	Adversarial security testing by authorized testers	Managed — linked to releases	TEST maturity gate

6.3 Test Management

The Test Management system provides centralized management of test cases, plans, suites, and results across all Solutions and environments. Test cases are version-controlled and linked to requirements, features, and Solution components.

- AI-driven test case generation synthesizes test scenarios from specifications and historical defect data
- AI-driven prioritization ranks test cases by risk, coverage gap, and historical failure rate
- Self-healing test capabilities detect and repair brittle tests when interfaces evolve
- Test results feed automatically into the Benchmarking and Analytics services
- Test case traceability matrix links every case to its requirement, feature, component, and defect history

6.4 Defect Tracking

Defects discovered during any test phase are captured, categorized, and tracked through resolution. The Defect Tracking system is linked to version control, build management, and release management.

Defect Field	Description
Defect ID	Unique identifier assigned at creation
Title	Short, descriptive summary of the defect
Severity	Critical High Medium Low — impact on Solution functionality and stability
Priority	P1 P2 P3 P4 — urgency of resolution relative to release schedule
Status	Open In Progress Fixed Verified Closed Won't Fix
Build Reference	The build version and number in which the defect was discovered
Commit Reference	The source commit(s) associated with the defect introduction
Test Case Reference	The test case that detected the defect
Assignee	The developer or team assigned to resolve the defect
Resolution	Description of the fix applied; linked to the resolving commit
Verification Date	Date and tester who verified the fix in the target environment

6.5 Prototyping & Sandbox Management

Prototyping environments in qala are first-class managed entities. They are isolated, governed, and integrated with the Solution Registry to ensure that prototype learnings feed back into the Solution Model and Solution Book.

- Automated provisioning of isolated sandbox environments for proof-of-concept (POC) work
- Prototypes are linked to parent Solutions, features, and projects in the Registry
- Prototype feedback is captured and analyzed automatically; AI summarizes findings and recommends next steps in the Solution Model
- Sandbox environments are ephemeral by default; promotion to persistent environments requires governance review
- Resource quotas and time limits are enforced on sandbox environments to control cost and prevent sprawl

7. Solution Factory (SF)

A Solution Factory (SF) is a coordinated, networked collection of Solution Development Environments (SDEs) that produces Solutions in a consistent, repeatable, and scalable way. The factory provides the production infrastructure — hermetic builds, CI/CD pipelines, artifact management, and governance controls — that transforms Solution Models into deployed Solutions. qala itself is the Root Solution Factory.

7.1 Factory Structure

Factory Element	Description
Solution Factory	Top-level coordination entity; owns a collection of networked SDEs and a governed solution portfolio
Solution Development Environment (SDE)	The atomic operational unit — fully specified in Section 8
Solution Network	The interconnected, orchestrated chain of SDEs operated by this Factory
Build System	Hermetic, reproducible build infrastructure for all Solutions produced by the Factory
CI/CD System	Automated integration, testing, and deployment pipelines spanning all factory SDEs
Artifact Repository	Versioned storage for all build outputs: binaries, containers, packages, and libraries
Solution Registry	Platform-level catalog of all Solutions, SDEs, and artifacts within this Factory
Solution Portfolio	The governed collection of Solutions produced and managed by this Factory
Governance Layer	Policies, templates, standards, and approval workflows applied to all factory output
Communications Module	Notification, alerting, and messaging infrastructure for all factory participants

7.2 Factory Hierarchy

qala itself is the Root Solution Factory. It produces tiered, hierarchical child factories that inherit governance, tooling, and pipeline configurations from their parent while maintaining the ability to specialize and extend.

qala (Root SF) → Enterprise Factory → Domain Factory → Team Factory → Project SDE → Developer SDE

Factory Tier	Scope	Governance Model	Typical Size
Root Factory (qala)	Operates all child factories; platform-level governance	Full platform governance — immutable root policies	Platform-wide
Enterprise Factory	Multi-team, multi-product factory with full governance suite	Inherits root + enterprise policy customization	Thousands of solutions
Domain Factory	Domain or business unit scoped factory	Inherits enterprise + domain-level standards	Hundreds of solutions
Team Factory	Single-team factory scoped to a product or service line	Inherits domain + team-level overrides	Tens of solutions
Personal Factory	Single-user factory for personal and hobby projects	Lightweight governance; owner is sole approver	Individual solutions

7.3 Factory Orchestration

Within a Solution Factory, SDEs are networked and orchestrated through the following mechanisms:

- Centralized workflow automation coordinating build, test, release, and deployment pipelines across SDEs
- Shared artifact and package repositories accessible to all environments within a factory
- Standardized event streaming (Apache Kafka) enabling loose-coupled communication between services and environments
- Policy enforcement ensuring all SDEs in a factory adhere to governance, security, and quality standards
- Coordinated CI/CD pipelines spanning multiple SDEs and environment tiers
- Solution Network: the interconnected chain of SDEs forming the factory's production network
- Solution Registry: authoritative catalog of all Solutions, artifacts, and environments within the factory
- Solution Portfolio: the governed aggregate view of all Solutions produced by the factory

7.4 Factory Service Categories

Service Category	Responsibilities
Environment Management	Provisioning, cloning, teardown, monitoring, backup, archival, and version control of SDEs
CI/CD & Build	Hermetic build execution, pipeline orchestration, artifact generation and signing
Testing & QA	Test case management, automated execution, defect tracking, testbed analytics
Release Management	Versioned release packaging, approval workflows, deployment automation, rollback capability
Artifact & Package Mgmt	Binary storage, dependency management, third-party tool integrations, supply-chain verification
Security & Compliance	SAST/DAST/IAST/SCA, vulnerability scanning, secrets management, audit logging, SDE quarantine
Benchmarking & Analytics	Performance, quality, and process metrics with dashboards, alerts, and cross-solution comparison
AI & Recommendations	Predictive insights, optimization suggestions, anomaly detection, self-healing automation
Governance & Standards	Policies, templates, playbooks, standards enforcement, ownership management, maturity gate control
Data Platform	Event streaming, data pipelines, warehouse, MDM, data lineage, and analytics aggregation
Communications	Notification dispatch, alerting, messaging, communication channel management

8. Solution Development Environment (SDE)

The Solution Development Environment (SDE) is the atomic operational unit of qala. Every developer, team, or automated process works within an SDE. SDEs are solution-agnostic by design — they define the conditions for creating, maintaining, and operating solutions, not the solutions themselves. One SDE can support a collection of solutions simultaneously.

8.1 SDE Properties

Property	Description
Deployable	Can be deployed to a platform, cloud environment, on-premises server, or physical machine
Configurable	All parameters, settings, and options are externally definable without modifying environment code
Distributable	Can be shared, cloned, replicated, and distributed across teams, organizations, or sites
Version-Controlled	Full history of environment state, configuration, and tooling; supports snapshots and rollbacks
Composable	Multiple SDEs can be combined, linked, or networked within a Solution Factory
Scalable	Supports single-user and enterprise-scale workloads without model change; resources scale to demand
Releasable	SDEs can be versioned and distributed as sealed, reproducible environment packages
Solution-Agnostic	Defines conditions for creation and operation; not tied to any specific solution type or domain

8.2 SDE Internal Architecture

8.2.1 Solution Configuration

Configuration Element	Description
Solution Version	The version of the Solution this SDE is configured to produce; pinned in the SDE configuration record
Solution Component References	References to each component, with component-level version pinning

Configuration Element	Description
Solution Part References	References to each part, with part-level version and vendor specification
Environment Variables	Runtime configuration values, secrets references, and parameter overrides; secrets never stored in plaintext
Configuration Files	Versioned, structured config files (YAML, TOML, JSON, ENV); stored in configuration repository
Settings / Parameters / Options	All SDE-level settings governing behavior, tooling, and runtime execution
User Preferences & Profiles	Developer-specific preferences, IDE bindings, and workspace layouts; portable across environments

8.2.2 Solution Model Elements

Model Element	Description
Solution Blueprint	Formal structural specification defining architecture and interfaces within this SDE's scope
Solution Design	Visual and functional design artifacts linked to this SDE's solution scope
Solution Architecture	System-level topology, service maps, data flow diagrams, and integration patterns
Solution Mockup	Static interface representations for stakeholder review and sign-off
Solution Prototype	Working POC or partial implementation for validation and feedback collection

8.2.3 Solution Environments Within SDE

Environment	Purpose	Characteristics
Sandbox Environment	Open-ended exploration and experimentation	No stability guarantees; ephemeral; no governance gates; isolated from production data
Build / Assembly Environment	Hermetic build and solution assembly	Isolated; reproducible; dependency-locked; attested; ephemeral per build
Test Environment	Full verification and validation	Isolated clone of production topology; auto-provisioned per test run; torn down after
Release Environment	Pre-release staging and CM sign-off	Production-equivalent; immutable after CM gate; DAST-scanned; performance-benchmarked

8.2.4 Solution Build Elements

Build Element	Description
Solution Build	A discrete, versioned output of the build pipeline for a Solution
Solution Build Version	Semantic version of this build output (MAJOR.MINOR.PATCH-BUILD)
Solution Build Maturity	Maturity level at time of build: SANDBOX DEV NIGHTLY TEST CM
Solution Build Number	Monotonically incrementing build sequence number within the version
Solution Build ID	Unique identifier for this specific build execution instance
Solution Build Timestamp	UTC timestamp of build initiation and completion
Solution Assembly	The composed, integrated output of all Components combined into a deliverable unit
Build Attestation	Signed SLSA provenance record linking this build to its verified inputs; immutable

8.2.5 Solution Toolbox

Tooling Element	Description
Tool	Atomic unit: a single executable, library, or service performing a specific function
Toolset	A named collection of related Tools serving a common purpose (e.g., 'Go Build Toolset')
Toolkit	A curated collection of Toolsets covering a full domain of practice (e.g., 'Backend Development Toolkit')
Toolchain	A linked, ordered sequence of Tools: each tool's output feeds the next (e.g., compile → link → sign → package)
Tool Suite	A governed, platform-approved set of Toolkits for a given Solution type or organizational standard
IDE Integrations	VSCode, JetBrains, and other developer environment plugins and extensions; versioned and qualified
3rd-Party Integrations	Vendor tool integrations managed, versioned, and qualified within the SDE; subject to Vendor Registry governance

8.2.6 Content Management System (CMS) Within SDE

Every SDE includes an integrated Content Management System for organizing all solution knowledge, documentation, and reference material:

- Solution documentation, charters, design documents, and specification files
- Playbooks, blueprints, templates, and standards references
- Version-controlled file management with full history and diff capabilities
- Workspace organization: folders, binders, collections, and directories
- Search and discovery across all content within the SDE
- AI-assisted content suggestions: documentation structure, template selection, and standards compliance

8.2.7 Communications & Networking Module

Each SDE includes a communications and networking module providing connections to the broader factory ecosystem:

- Connections to external services, APIs, and data sources
- Event publication and subscription to the platform-wide event bus
- Notification channels: email, message, alert, and announcement routing
- Communication rooms: gigs, consultations, bookings, tasks, jobs, contracts, offers, deals, requests, proposals
- Community and team collaboration spaces

8.3 SDE Lifecycle States

State	Trigger	Description	Permitted Operations
Provisioning	SDE creation request	Environment being assembled from template or definition	Read-only status monitoring
Active	Provisioning complete	Fully operational; all development and build activities permitted	All operations
Snapshotted	User or CI action	State captured; environment continues to run normally	All operations; snapshot available for rollback
Suspended	Inactivity or admin action	Resources released; state preserved for resumption	Resume, archive, terminate
Rolled Back	Restore request	Environment restored to a prior snapshot version	All operations after rollback completes
Archived	Retention policy or admin	Moved to long-term storage; retrievable on demand	Restore, metadata query
Terminated	Explicit deletion	Permanently decommissioned; backups retained per policy	Metadata query only

8.4 SDE Maturity

Each SDE carries a maturity classification aligned to the Solution it is producing. The SDE maturity gates control which operations are permitted within the environment — only CM-maturity SDEs may produce immutable release artifacts. SDE maturity cannot exceed the maturity of the parent Solution Factory's governance tier.

8.5 SDE Backup, Recovery & Archiving

Operation	Description	Triggering Conditions
Snapshot	Point-in-time capture of full SDE state; supports rollback and branching	User-triggered, CI action, or scheduled policy
Rollback	Restore SDE to a prior snapshot version; CI/CD pipelines resume immediately after restore	User request or automated recovery procedure
Backup	Scheduled or event-driven backup: full or incremental; stored in versioned secure storage (on-prem or cloud)	Scheduled policy, pre-deployment gate, or significant state change
Clone	Create an independent copy of an SDE for parallel development, benchmarking, or environment comparison	User request or automation
Archive	Move inactive or legacy SDEs to long-term storage with retention policy enforcement	Inactivity threshold, policy schedule, or admin action
Restore	Rehydrate an archived or backed-up SDE to an active state; compatibility validated automatically	User request or disaster recovery procedure

8.5.1 Backup Data Model

Field	Description
Backup ID	Unique identifier for this backup record
SDE ID	Reference to the SDE that was backed up
Environment Version / Snapshot	Version and snapshot number captured in this backup
Timestamp	UTC timestamp of backup initiation and completion
Backup Type	Full Incremental
Storage Location	On-premises path or cloud storage URI

Field	Description
Status	Success Failed Pending Corrupted

9. Hermetic Build System

qala mandates hermetic build environments for all solution artifacts. Hermetic builds guarantee that the same source code and inputs will always produce the same outputs, regardless of when or where the build is executed. This is the foundational contract of the qala build system — every artifact is reproducible, every build is auditable, and every output is signed.

9.1 Hermetic Environment Design Principles

- Fully isolated: no access to external network resources, host file system, or ambient credentials during build execution
- Reproducible: identical inputs always produce byte-identical outputs; non-determinism is a build failure
- Environment-as-code: all build environment definitions are version-controlled and immutable at build time
- Containerized: each build executes in a purpose-built, ephemeral container spun up and destroyed per run
- Dependency locking: all dependency versions are pinned in a lock file and cryptographically verified before execution begins
- Build attestation: every build produces a signed SLSA provenance record linking output artifacts to their verified inputs
- Immutable environments: no state persists between builds unless explicitly captured as a versioned artifact
- Supply chain verification: all tool and dependency binaries are verified against cryptographic hashes before use

9.2 Build Attestation & Signatures

Every build artifact produced by qala is accompanied by a signed attestation record conforming to the SLSA (Supply chain Levels for Software Artifacts) framework. Attestations are stored immutably alongside artifacts and are verifiable at any point in the artifact lifecycle. Deployment pipelines may be configured to reject artifacts without valid attestations.

Attestation Field	Description
Build ID	Unique identifier for this build execution instance
Source Commit Hash	Cryptographic hash of the source code at build time (SHA-256)
Environment Hash	Hash of the complete build environment definition (Dockerfile, Nix expression, or equivalent)
Dependency Manifest	Full lock file of all resolved dependencies with pinned versions and SHA-256 hashes
Build Timestamp	UTC timestamp of build initiation and completion

Attestation Field	Description
Artifact Hash	SHA-256 of each produced artifact binary; verified at deployment
Builder Identity	Signing key identifier of the CI/CD agent or user who triggered the build
SLSA Provenance Level	SLSA Level 1 2 3 — indicating the degree of build integrity guarantee
Signature	Cryptographic signature (sigstore/cosign or equivalent) over all attestation fields
Verification URL	URI at which this attestation can be independently verified

9.3 Standard Hermetic Build Pipeline

A standard hermetic build pipeline in qala consists of the following ordered stages. Each stage is executed within the isolated build environment and produces a verified output before the next stage begins.

1. Source Fetch — checked out at exact commit hash, verified against SCM signature
2. Environment Provision — hermetic container instantiated from the locked, version-controlled environment definition
3. Dependency Resolution — all dependencies resolved from internal mirror, lock file verified, hashes confirmed
4. Compile / Build — source compiled within the isolated environment; no external network access
5. Unit Test — automated unit test suite executed within the build environment; results captured as artifacts
6. Security Scan (SAST) — static analysis applied to source code and dependency graph
7. Code Quality Gate — linting, complexity, coverage, and standards compliance checks applied
8. Artifact Package — output binaries packaged, hashed, and prepared for signing
9. Attestation Generation — SLSA build provenance record created and cryptographically signed
10. Artifact Publication — packages and attestation stored in the Artifact & Package Manager
11. Environment Teardown — hermetic container destroyed; no state persists outside captured artifacts

9.4 Build Configuration as Code

All build pipeline definitions, environment specifications, and dependency lock files are version-controlled alongside the source code they build. This means that every commit to the repository carries with it the complete definition of how it should be built, tested, and packaged — enabling full reproducibility at any point in the version history.

10. CI/CD & Configuration Management

qala integrates Continuous Integration, Continuous Deployment, and Configuration Management (CI+CD+CM) as tightly coupled, first-class platform capabilities. Every code commit, configuration change, and environment mutation flows through this unified system.

10.1 Continuous Integration (CI)

- The CI system automatically triggers on every commit, pull request, or merge event from the version control system. CI pipelines are always executed in hermetic, ephemeral build environments as described in Section 9.
- Pipelines are automatically triggered on commit, pull request, or merge events
 - Every pipeline run is executed in a hermetic, ephemeral build environment
 - Code quality gates enforce linting, static analysis, test coverage thresholds, and complexity limits
 - Security policies (SAST, SCA, dependency scanning) are mandatory gates; failures block artifact promotion
 - Pipeline definitions are version-controlled alongside source code as part of the environment-as-code principle
 - AI-driven pipeline optimization identifies bottlenecks and recommends restructuring for faster feedback loops
 - Parallel stage execution where dependency graph permits, minimizing total pipeline duration

10.2 CI/CD/CM Pipeline Stages

Pipeline Stage	Trigger	Actions	Output
CI — Integration	Every commit / pull request	Build, unit test, SAST scan, SCA, code quality gate, artifact generation	Verified build artifact + attestation
CD — Delivery	CI green + merge to main	Integration tests, system tests, DAST scan, staging deployment, performance benchmarks	Staging-validated artifact
CD — Deployment	CD green + maturity gate	Automated promotion: canary, blue/green, or rolling deployment to target environment	Running Solution instance
CM — Configuration Mgmt	CM board approval	Immutable release tagging, audit trail, environment lock-down, governance sign-off	CM-qualified release record

10.3 Deployment Strategies

Strategy	Description	Best For
Blue/Green	Two parallel environments; traffic switched atomically on validation; instant rollback by switching back	High-availability services; zero-downtime releases
Canary	Progressive traffic shift to new version; automatic rollback on performance or error rate degradation	Risk-sensitive releases; gradual validation under real traffic
Rolling	Incremental instance replacement; maintains capacity and availability throughout deployment	Stateless services; large-scale deployments
Recreate	Full teardown and redeploy; acceptable for non-critical or stateless services	Low-traffic services; development environments
Feature Flag	Code deployed to all instances; features gated by configuration flags per user segment	A/B testing; phased rollouts; emergency kill-switches

10.4 Configuration Management (CM)

Configuration Management in qala provides governed, version-controlled control of all environment and system configuration. CM is the highest maturity stage and represents the production-qualified, immutable state of a Solution.

- All environment configurations are version-controlled and stored in a central configuration repository
- Configuration changes trigger their own pipelines: validate, apply, verify, and record
- Configuration drift is detected continuously; automatic remediation or alerts are raised when drift exceeds threshold
- Environment templates are parameterized by technology stack, project, and release context
- Configurations are fully auditable: every change is logged with author, timestamp, change justification, and impact assessment
- Secrets are excluded from configuration files and managed through the Secrets Vault exclusively (see Section 17)

10.5 Version Control Management

qala provides a unified version control management layer that integrates with industry-standard SCM systems (Git-compatible) while adding platform-level capabilities:

- Branch policies, merge requirements, and approval workflows enforced platform-wide across all repositories
- Automatic linking of commits to solutions, projects, bugs, features, and releases
- Semantic versioning enforced for all solution artifacts and SDE configurations
- Change impact analysis: automatically identify which solutions, tests, and deployments are affected by a given commit

- AI-assisted code review: suggestions for code quality, security compliance, and standards adherence
- Dependency graph tracking: understand propagation of changes across the solution dependency tree

10.6 Change Control Management

All significant changes to solutions, environments, or infrastructure follow a governed change control process:

- Change requests are logged with impact assessment, risk rating, rollback plan, and linked business justification
- Approval workflows are configurable by change type, risk level, environment tier, and organizational policy
- Post-implementation reviews are automatically scheduled and linked to change records with outcome data
- Emergency change procedures support expedited approval for critical fixes with mandatory post-hoc review
- All changes are traceable to the business objective, project, incident, or feature request that motivated them

11. Solution Tooling Management

Solution Tooling defines the full hierarchy of tools available within a Solution Development Environment. Tooling is version-controlled, vendor-managed, cryptographically verified, and composable — from individual atomic tools up through toolsets, toolkits, toolchains, and tool suites. The Tool Registry is the authoritative catalog for all tools approved for use on the qala platform.

11.1 Tooling Hierarchy

Level	Description	Example
Tool	Atomic unit: a single executable, library, or service performing a specific function	rustc, go, docker, kubectl, terraform, golint, npm
Toolset	A named collection of related Tools serving a common purpose	'Go Build Toolset': go, gofmt, golint, gotest, go vet
Toolkit	A curated collection of Toolsets covering a full domain of practice	'Backend Development Toolkit': build, test, lint, container toolsets
Toolchain	A linked, ordered sequence of Tools where each tool's output feeds the next	compile → link → sign → package → attest → publish
Tool Suite	A governed, platform-approved set of Toolkits for a given Solution type	'Rust Microservice Suite': compilation, testing, security, deployment toolkits

11.2 Tool Record

Tool Field	Description
Tool ID	Unique identifier in the qala Tool Registry
Tool Name	Human-readable name (e.g., rustc, docker, terraform)
Tool Version	Pinned semantic version used within this SDE; version pinning is mandatory
Tool Type	compiler linter formatter build-system test-runner container IaC IDE VCS scanner signing other
Vendor / Source	Tool publisher or open-source project origin
License	License type (MIT, Apache 2.0, BSD, GPL, proprietary, etc.)
Toolset Membership	Which Toolsets include this Tool
Toolchain Position	Position in any Toolchains this Tool participates in

Tool Field	Description
Verification Hash	SHA-256 of the tool binary for supply-chain verification; checked at every environment provision
Qualification Status	Approved Provisional Deprecated Blocked
Security Advisories	Link to active CVEs or security advisories affecting this tool version

11.3 Tool Version Control

All tools used within qala SDEs are subject to platform-level version control governance. Tool versions are pinned in SDE configuration records, stored in the Tool Registry, and verified cryptographically at every environment provisioning event. Tool upgrades follow a defined upgrade workflow: advisory detection → impact assessment → validation in sandbox → gradual rollout → registry promotion.

11.4 Third-Party Tool Integration Management

Third-party vendor tool integrations are managed as first-class entities within the platform. Integration governance covers onboarding, configuration, security qualification, licensing compliance, and lifecycle management.

- Vendor tools are onboarded through the Vendor Registry qualification workflow
- All third-party binaries are verified against cryptographic hashes from the vendor's official distribution
- Tool integrations are configuration-managed: every configuration is version-controlled and auditable
- License compliance is tracked automatically; license changes trigger notification and review workflows
- Deprecated or blocked tools are flagged with migration paths and sunset timelines

12. Benchmarking & Performance Analysis

qala integrates multi-dimensional benchmarking across performance, quality, and process dimensions. Benchmarking data is collected continuously and aggregated into dashboards, alerts, and AI-driven recommendations. Benchmarking is embedded in the CI/CD pipeline as a mandatory gate for Solutions advancing to TEST and CM maturity.

12.1 Performance Benchmarking

- CPU, memory, disk I/O, and network utilization per Solution, Component, and environment
- Response time (p50, p90, p95, p99) and throughput under nominal and stress load conditions
- Scalability profiling: behavior under progressive load increases; horizontal and vertical scaling limits
- Automated performance tests executed in CI/CD on every release candidate at TEST maturity and above
- Cross-release and cross-environment comparison to identify regressions before production deployment
- Scenario benchmarking: simulate peak load, environment-specific conditions, and projected growth curves

12.2 Quality Benchmarking

- Code complexity, maintainability index, and duplication metrics per component and solution
- Linting and coding standard adherence scores with configurable thresholds per technology stack
- Test coverage: unit, integration, and system levels; mandatory minimum thresholds enforced as maturity gate criteria
- Defect density, severity distribution, and MTTR (Mean Time to Resolution) trends over time
- Technical debt tracking: identification, quantification, and amortization planning

12.3 Process Benchmarking

- Development velocity: story points or equivalent per sprint or iteration
- Deployment frequency and lead time from commit to production
- Release success rate, rollback frequency, and hotfix rate
- CI/CD pipeline duration and failure rates by stage and by solution
- Mean Time to Detect (MTTD) and MTTR for incidents linked to deployed solutions

12.4 Cross-Solution Benchmarking

The platform enables aggregate benchmarking across the entire factory portfolio — comparing metrics across projects, teams, applications, and products to identify high-performing patterns and recommend adoption in lower-performing areas.

- Compare metrics across projects, teams, applications, and products within a factory
- Identify high-performing patterns in build times, test coverage, deployment frequency, and defect density
- AI-driven benchmarking predicts potential performance or defect hotspots before they materialize
- Recommend adoption of high-performing practices to underperforming teams and solution areas

12.5 Benchmarking Dashboards

Audience	Dashboard Content
Executives	KPI trends, release velocity, quality posture, risk summary, portfolio health distribution
Architects	Code quality, technical debt, dependency health, system complexity, cross-service coupling metrics
QA & Release Managers	Test coverage, defect rates, pipeline health, deployment outcomes, rollback frequency
Developers	Personal SDE metrics, build times, code quality feedback, test results, debt items
Security Teams	Vulnerability scan results, threat density, SDE compliance status, policy adherence

13. Security & Privacy Management

Security and privacy are embedded into every layer of the qala platform — not appended as afterthoughts. Every SDE, pipeline, artifact, and data flow is subject to continuous security monitoring, policy enforcement, and automated scanning. The Security and Event Management (SEM) service is the platform-wide security coordination layer.

13.1 Security Controls Overview

Control	Description
RBAC	Role-Based Access Control enforced at every API endpoint, SDE operation, and administrative function
MFA	Multi-factor authentication required for all human users and privileged service accounts
SSO	Single Sign-On integration with enterprise identity providers (SAML 2.0, OIDC, OAuth 2.0)
mTLS	Mutual TLS for all service-to-service communication within the platform; no plaintext internal traffic
Secrets Vault	HashiCorp Vault or cloud-native KMS for all credentials, certificates, API keys, and secrets
SAST	Static Application Security Testing on every commit and build
DAST	Dynamic Application Security Testing on every build targeting TEST+ maturity
IAST	Interactive Application Security Testing during integration test stage execution
SCA	Software Composition Analysis: dependency and supply-chain vulnerability scanning
Container Scan	Base image and runtime container scanning before promotion to any environment
Build Attestation	SLSA provenance records signed for every build output; verified before deployment
Supply Chain Verification	All tool and dependency binaries verified against cryptographic hashes before use
Immutable Audit Logs	All platform actions recorded in tamper-evident, append-only audit logs per tenant
Data Encryption	Encryption at rest (AES-256) and in transit (TLS 1.3) mandatory for all solution and tenant data

Control	Description
Threat Detection	Real-time SEM service monitors for anomalous activity, policy violations, and attack patterns
SDE Quarantine	Compromised or suspicious SDEs automatically isolated by the SEM service; factory notified

13.2 Security Testing Integration

Test Type	Acronym	Integration Point	Blocking on Failure
Static Application Security Testing	SAST	Pre-commit hooks and CI pipeline build stage	Yes — blocks artifact promotion
Dynamic Application Security Testing	DAST	Post-deploy automated scanning in staging environments	Yes — blocks CM promotion
Interactive Application Security Testing	IAST	Instrumented test execution during integration stage	Yes — critical findings block
Software Composition Analysis	SCA	Dependency resolution stage; scans all resolved packages	Yes — known-vulnerable packages blocked
Container Scanning	—	Base image scanning and runtime container scanning before promotion	Yes — critical vulnerabilities block
Penetration Testing	Pentest	Managed via Test Management; required for TEST → CM promotion	Yes — P1 findings block CM gate

13.3 Vulnerability Management

- Continuous vulnerability scanning of code, dependencies, containers, and infrastructure components
- Risk scoring by severity (CVSS), exploitability, and asset criticality; prioritized remediation queue
- Automated patching recommendations with impact analysis before application
- Remediation tracking: vulnerabilities linked to tickets, owners, and SLA targets by severity tier
- High-severity findings (CVSS 7.0+) trigger immediate alerts and block pipeline promotion

- Vulnerability trends tracked over time; MTTD and MTTR metrics reported to security dashboards

13.4 Threat Modeling & Risk Assessment

- Threat models created and versioned per Solution and Application during the design phase
- Attack surface mapping updated automatically as Solution structure changes (new components, interfaces, dependencies)
- Risks assigned severity, probability, and mitigation plans; linked to governance records
- Threat intelligence feeds keep the platform current with emerging attack patterns and CVEs
- AI-assisted threat correlation: the SEM service correlates security events to identify attack patterns and risks

13.5 Privacy Management

- PII identification and classification across all data flows and stores; automated detection via AI classifiers
- Data masking, anonymization, and encryption policies enforced per data classification tier
- Retention and deletion policies with automated enforcement and audit trail
- Audit logs of all access to and modification of sensitive and personally identifiable data
- Compliance dashboards for GDPR, CCPA, ISO 27001, SOC 2, and configurable regulatory frameworks

13.6 Secrets Management

- All credentials, API keys, certificates, and secrets stored in a hardware-backed vault (HashiCorp Vault or cloud KMS)
- Secrets are never embedded in source code, configuration files, environment variables, or logs
- Dynamic secret generation with short TTLs for ephemeral environments; secrets expire automatically
- Access to secrets is audited, role-gated, and revocable in real time
- Secret rotation is automated; rotation events are logged and propagated to all dependent services

14. Release & Deployment Management

qala manages the complete release lifecycle from packaging through production deployment and post-release monitoring. Release management is tightly integrated with testing, security, governance, and configuration management to ensure only validated, compliant artifacts reach production environments.

14.1 Artifact & Package Management

The Artifact & Package Management service provides a centralized, versioned repository for all compiled outputs: libraries, binaries, containers, packages, and Solution artifacts.

- Semantic versioning enforced on all artifacts with full dependency graph tracking
- Artifacts are immutable after publication; updates always produce a new versioned artifact
- Third-party vendor packages are proxied through an internal mirror for integrity verification and availability
- Artifact promotion gates: artifacts must pass security and quality checks before promotion to higher environment tiers
- Rollback is achieved by redeploying a prior artifact version — no recompilation required
- Build attestation linked to every artifact: provenance is verifiable at any point in the artifact lifecycle

14.2 Release Pipeline

Build → Unit Test → Security Scan → Package → Integration Test → Performance Test → Staging Deploy → Acceptance Test → CM Gate → Production Deploy → Post-Deploy Verification

Each stage has configurable pass/fail criteria. Failed stages halt progression and trigger notifications. Stages may run in parallel where the dependency graph permits.

14.3 Environment Tiers

Tier	Purpose	Promotion Gate	Maturity Alignment
Development	Active development and local testing	Unit tests passing	SANDBOX / DEV
Integration	Cross-service and cross-component testing	Integration test suite passing	NIGHTLY
Staging	Production-equivalent pre-release validation	Performance + acceptance tests + DAST	TEST

Tier	Purpose	Promotion Gate	Maturity Alignment
Production	Live environment serving end users	Release manager approval + all gates + CM sign-off	CM

14.4 Solution Release Management

Solution Release Management governs the end-to-end process of versioning, packaging, approving, and distributing a Solution across its distribution channels.

- Solution Release Train: a scheduled cadence for releasing groups of solutions together with coordinated testing
- Solution Rollout: controlled distribution of a release to progressively larger audiences or environments
- Release Changelog: human-readable record of all changes included in each release version
- Release Notes: stakeholder-facing communication of new capabilities, fixes, and deprecations
- Release Approval Workflow: multi-party sign-off process for CM-stage promotions
- Distribution Channels: communication and logistics paths through which a Solution release is delivered to consumers

14.5 Solution Distribution & Supply Chain

qala manages the full distribution lifecycle of Solutions, including supply chain integrity, logistics coordination, and inventory management for physical artifacts and digital deliverables.

- Solution Artifact Management System: inventory management, binary management, and asset tracking
- Solution Supply Chain: traceability of all components and parts from vendor to final deliverable
- Solution Inventory Management System: real-time tracking of all released artifacts across environments and tiers
- Solution Warehouse: centralized repository for all released solution packages across all versions

15. Governance, Standards & Lifecycle Management

qala embeds governance as a native platform capability. Every Solution, environment, and artifact exists within a governed context that defines ownership, policies, standards, and lifecycle rules. Governance is enforced through machine-readable policies, automated gate criteria, and human approval workflows.

15.1 Governance Instruments

Instrument	Description
Policy	A declarative rule constraining behavior at the platform, factory, or solution level; machine-readable and version-controlled
Standard	A defined approach or specification that Solutions must conform to (e.g., API versioning, naming conventions, coding standards)
Template	A pre-approved scaffold for common Solution types, SDE configurations, or workflow definitions; AI-recommended by context
Playbook	A step-by-step guide for executing a defined process (e.g., Incident Response, Release Procedure, Onboarding)
Gate Criteria	Specific, measurable conditions that must be satisfied for a Solution to advance between maturity stages
Audit Log	Immutable, tamper-evident record of all actions taken on Solutions, SDEs, Factories, and platform resources
Ownership Record	Documented assignment of accountability for every Solution, Component, SDE, and Factory
Exception Record	Documented, time-limited approval for deviating from a policy; requires justification and renewal

15.2 Maturity Gate Criteria

Maturity Promotion	Required Gate Criteria
SANDBOX → DEV	Solution record created in Registry; SDE provisioned; owner assigned; value proposition documented
DEV → NIGHTLY	CI pipeline configured and green; basic build succeeds; unit tests passing; code quality baseline established
NIGHTLY → TEST	All unit and integration tests passing on main branch; SAST scan clean; SCA scan clean; no P1 defects open; performance baseline captured

Maturity Promotion	Required Gate Criteria
TEST → CM	All test suites passing; DAST clean; container scan clean; performance benchmarks met; pentest completed; governance board sign-off; build attestation generated; CM board approval

15.3 SDE Governance & Lifecycle State Management

SDEs are governed through their full lifecycle with state transitions subject to policy validation. SDE governance is enforced independently of Solution governance but is aligned to the maturity model.

- Templates define approved SDE configurations; deviations require justification and approval
- Lifecycle state transitions (e.g., Active → Archived) are audited and may require multi-party approval
- Resource quotas and cost allocations are tracked per SDE and reported to owners and factory administrators
- Compliance scanning verifies SDE configurations against security baselines on a scheduled basis
- Orphaned SDEs (no active owner or active solution) are flagged and escalated for disposition

15.4 Standards, Templates & Playbook Library

The platform maintains a central, governed library of approved project templates, environment definitions, pipeline configurations, and code scaffolds. Templates are versioned, categorized by technology stack and solution type, and recommended by AI based on context.

- AI-recommended templates: the platform suggests the most appropriate template based on solution type, factory tier, and historical success patterns
- Deviations from standard templates are tracked as technical debt items requiring justification and amortization planning
- Standards are reviewed on a defined cadence and updated to reflect evolving best practices
- Playbooks are linked to specific maturity stages and governance events for just-in-time delivery to practitioners

15.5 Solution Ownership System

- Every Solution has a primary owner — a person or team — responsible for quality, compliance, roadmap, and lifecycle decisions
- Contributing owners may be defined for shared responsibilities: security, performance, documentation, and integration
- Ownership transfers are tracked, require formal acknowledgment from both parties, and are logged in the audit record
- Orphaned solutions (no active owner) are flagged and escalated to factory administrators for disposition

- Ownership dashboards show the complete portfolio of each owner's assigned solutions, their maturity distribution, and outstanding obligations

16. Solution Book

The Solution Book is the complete knowledge and documentation repository for a Solution. It consolidates all records, plans, structures, schedules, and organizational artifacts that govern and describe a Solution from inception through retirement. Every Solution has exactly one Solution Book; the Book is versioned alongside the Solution and constitutes the authoritative knowledge record of that Solution.

16.1 Solution Book Structure

Book Element	Description
Charter	The foundational document: purpose, scope, goals, success criteria, and owner sign-off
Notes	Running log of decisions, observations, meeting notes, and ad hoc records
Parts	The canonical parts inventory: all Solution Parts referenced by this Solution with sourcing information
Vendors	Vendor records and qualification status for all suppliers contributing to this Solution
Binders	Logical groupings of related documents, assets, and content within the Book
Directories	Navigational indexes for all files, documents, and content in the Book
Lists	General-purpose ordered or unordered lists: requirements, features, risks, decisions, action items
Collections	Named sets of related items: asset collections, reference sets, content libraries
Schedules	Planned timeline of activities, milestones, and deliverable dates with status tracking
Timelines	Visual chronological representations of Solution history and planned events
Work Packages	Bounded units of work with defined scope, owner, deliverable, and effort estimate
Work Breakdown Structures (WBSs)	Hierarchical decomposition of all work required to deliver the Solution
Playbooks	Step-by-step guides for repeatable processes: release, incident response, onboarding, scaling

Book Element	Description
Roadmaps	Forward-looking plans: feature roadmaps, version plans, and strategic milestones
Business Model / Plan	Value proposition, market positioning, revenue model, and strategic plan for the Solution

17. Solution Package

A Solution Package is the bundled, versioned, distributable form of a Solution. It encapsulates the Solution artifact, its metadata, configuration manifests, dependencies, and installation and deployment instructions into a single distributable unit that can be stored, transferred, deployed, or published.

17.1 Package Structure

Package Element	Description
Package ID	Unique identifier for this specific package version
Solution Reference	The Solution and Solution version this package represents
Package Version	The distribution version of this package (may differ from Solution version in re-packaging scenarios)
Artifact Manifest	List of all artifacts, binaries, containers, and assets included in this package
Dependency Manifest	All declared dependencies with pinned versions and SHA-256 verification hashes
Configuration Template	Default configuration values and required parameter definitions for deployment
Installation / Deployment Instructions	Steps to deploy or install this package into a target environment
Build Attestation	Signed SLSA provenance record linking this package to its verified build inputs
License	Applicable license(s) governing use and distribution of this package
Changelog	Human-readable record of changes from the previous package version
Compatibility Matrix	Declared compatibility with target environments, operating systems, and dependency versions

18. Solution Compositions: Chain, Set, Kit & Tool Solution

Higher-order Solution constructs allow multiple Solutions to be grouped, linked, or composed into coherent offerings. These constructs operate at the Solution Registry and Factory Portfolio level, enabling complex solution architectures to be represented, governed, and deployed as unified entities.

Construct	Description	Typical Use
Solution Chain	An ordered, dependency-linked sequence of Solutions where the output of one feeds the next as input	End-to-end value chains; pipeline compositions; platform stacks; data pipelines
Solution Set	An unordered collection of Solutions deployed or managed together without strict dependency ordering	Product suites; feature bundles; release groups; coordinated deployments
Solution Kit	A curated, pre-configured bundle of Solutions, tools, and configurations designed to solve a specific class of problem	Starter kits; reference implementations; accelerators; domain-specific bundles
Tool Solution	A Solution specifically classified as a tool — consumed by other Solutions or by SDEs as a tooling dependency	Compilers, linters, build systems, test runners, CLI utilities, internal libraries

18.1 Solution Configure-Price-Quote (CPQ)

For market-facing Solutions of type Product, Service, or Platform, qala provides a Configure-Price-Quote module that enables structured modeling of solution offerings, pricing tiers, and customer-facing configurations. The CPQ module links to the Solution's feature list, component structure, and vendor catalog to produce accurate, traceable quotes and offering configurations.

- **Solution Offerings:** defined bundles of features and capabilities presented to customers or internal consumers
- **Configuration Management:** structured selection of optional components, features, and parameters
- **Pricing Model:** cost structures linked to component usage, license tiers, and deployment scale
- **Quote Generation:** automated assembly of offering configurations into formal quotes with version and validity tracking

19. Solution Orchestration

Solution Orchestration defines how the work of creating, delivering, and operating a Solution is organized into structured, traceable workflows and tasks. Orchestration connects Solution Models to Solution Factories, coordinating the sequence and parallelism of all factory activities from ideation to deployment.

19.1 Orchestration Hierarchy

Level	Description
Solution Orchestration	Top-level coordination model for all work associated with a Solution; links to the Solution Book and Solution Factory
Workflow	A named, directed acyclic graph (DAG) of Tasks representing a complete end-to-end process
Task	The atomic unit of orchestrated work: an action with defined inputs, outputs, assignee, and completion criteria

19.2 Workflow Types

Workflow Type	Description	Trigger
Build Workflow	Executes the hermetic build pipeline for a Solution or Component	Commit, PR merge, or scheduled trigger
Test Workflow	Runs a defined test suite against a Solution build	Build completion or scheduled run
Release Workflow	Orchestrates build → test → stage → approve → deploy for a Solution release	Release trigger or maturity promotion request
Deployment Workflow	Executes the deployment of a Solution artifact to a target environment	Release approval or automated promotion gate
Rollback Workflow	Restores a previous Solution version in a target environment	Deployment failure or manual trigger
Provisioning Workflow	Creates and configures a new SDE from a template	User request or automation
Governance Workflow	Routes a Solution through review, approval, and CM promotion gates	Maturity promotion request

Workflow Type	Description	Trigger
Data Pipeline Workflow	Executes extract, transform, and load operations for Solution data	Scheduled trigger or event
Security Workflow	Triggers security scans, threat assessments, or SDE quarantine procedures	Security event, policy violation, or scheduled scan

19.3 Task Schema

Task Field	Description
Task ID	Unique identifier assigned at task creation
Task Name	Human-readable label describing the task's purpose
Workflow Reference	Parent workflow this Task belongs to
Task Type	build test deploy review approve notify script manual security data
Inputs	Data, artifacts, or conditions required to begin this Task
Outputs	Artifacts, state changes, or signals produced on Task completion
Assignee	User, team, service, or AI agent responsible for execution
Status	pending running succeeded failed skipped blocked cancelled
Dependencies	List of Task IDs that must complete successfully before this Task may begin
Timeout	Maximum allowed execution time before the Task is marked as failed
Retry Policy	Number of retries and backoff strategy on failure (immediate, linear, exponential)
SLA	Expected completion time; alerts triggered if exceeded

20. Solution Vendor Management

A Solution Vendor represents any external or internal entity that supplies components, parts, services, or capabilities to a Solution or Solution Factory. The Vendor Registry maintains a catalog of all qualified suppliers and their associated Parts and Components. Vendor management is integrated with supply chain verification, license compliance, and security advisory monitoring.

20.1 Vendor Record

Vendor Field	Description
Vendor ID	Unique identifier for this vendor in the qala Vendor Registry
Vendor Name	Legal or trade name of the supplier
Vendor Type	Hardware Supplier Software Library SaaS Provider Service Provider Internal Team Open Source Project
Contact Information	Primary contact person, email, and communication channels
Parts Catalog	List of Parts and Components this vendor supplies, with part numbers and versions
Qualification Status	Approved Provisional Deprecated Blocked
SLA / Agreement Reference	Link to the governing contract, license agreement, or SLA document
Lead Time	Typical delivery or provisioning time for this vendor's offerings
Security Advisories	Active CVEs or security advisories affecting any parts from this vendor
License Terms	License type(s) governing all parts supplied by this vendor; compliance status

20.2 Vendor Qualification Workflow

12. Submission: vendor submits Parts catalog, documentation, and certification evidence
13. Review: governance team validates compliance with quality, security, and licensing standards
14. Provisional Approval: vendor may be used in SANDBOX through TEST maturity stages under monitoring
15. Full Approval: vendor promoted to Approved status; Parts may be used in CM-stage Solutions
16. Ongoing Monitoring: SLA compliance, security advisories, and license changes tracked continuously; alerts on changes

21. Solution Registry

The Solution Registry is the authoritative catalog for all Solutions, Solution Models, Vendors, Tools, Factories, Artifacts, and Portfolios managed on the qala platform. It provides discovery, lineage tracking, governance assignment, and cross-reference services across the entire platform. Every object in qala has a registry entry.

21.1 Registry Catalogs

Registry Catalog	Contents
Solution Registry	All Solutions: type, version, maturity, owner, SDE reference, factory reference, feature list
Solution Models Registry	All Blueprints, Designs, Architectures, Mockups, and Prototypes; linked to parent Solutions
Vendor Registry	All approved Vendors: parts catalog, qualification status, SLA references, license terms
Tool Registry	All Tools: version, type, license, verification hash, toolset membership, qualification status
Factory Registry	All Solution Factories: SDE network, governance configuration, produced Solutions, portfolio
Artifact Registry	All build artifacts: binary, version, attestation record, storage location, promotion status
Portfolio Registry	All Solution Portfolios: composition, owner, factory association, maturity distribution
SDE Registry	All SDEs: status, maturity, owner, factory reference, snapshot history, resource utilization

22. Solution Portfolio

A Solution Portfolio is a governed collection of Solutions managed under a common Solution Factory or organizational unit. Portfolios provide aggregate visibility into solution health, maturity distribution, value delivery, and strategic alignment. Every Solution Factory maintains one or more Portfolios.

22.1 Portfolio Record

Portfolio Field	Description
Portfolio ID	Unique identifier
Portfolio Name	Human-readable name (e.g., 'Platform Engineering Portfolio', 'Consumer Products Portfolio')
Owner	The organizational unit or governance board accountable for this Portfolio
Factory Reference	The Solution Factory that produces the Solutions in this Portfolio
Solutions	The set of Solutions contained in this Portfolio, with maturity and version tracking
Maturity Distribution	Aggregate view of how Solutions are distributed across SANDBOX DEV NIGHTLY TEST CM stages
Value Chain Coverage	Which value chain stages are actively instrumented and measured for this Portfolio
Governance Status	Overall governance health: open gates, policy violations, pending approvals, orphaned solutions
Strategic Alignment	Link to organizational objectives, OKRs, or strategic initiatives this Portfolio supports

23. Solution Value Chain

qala provides a Solution Value Chain model that maps the end-to-end flow of value from inputs (resources, ideas, requirements) through to outcomes (deployed solutions, user value, business results). The value chain is tracked, measured, and optimized continuously. Every stage of the value chain is instrumented with metrics, events, and AI-driven insights.

23.1 Value Chain Stages

Stage	Key Activities	Output	Metrics
Ideation	Problem definition, goal setting, feasibility assessment, value proposition authoring	Solution Charter, value proposition statement	Time to charter, feasibility score
Design	Blueprint authoring, prototype creation, design review, mockup validation	Approved Solution Model	Design cycle time, review iterations
Build	Hermetic build, unit tests, CI pipeline execution, dependency resolution	Verified build artifact with attestation	Build duration, test pass rate, defect density
Test	Full test suite execution, defect resolution, security scan, benchmarking	Test-qualified artifact	Test coverage, MTTD, defect closure rate
Release	CM board review, version tagging, changelog, sign-off, package assembly	CM-approved release	Release cycle time, approval lead time
Deploy	Deployment workflow execution to target environment, post-deploy verification	Running Solution instance	Deployment duration, rollback rate, availability
Operate	Monitoring, observability, incident response, continuous optimization, patching	Stable operational Solution	MTTR, uptime, performance SLA compliance
Retire	Deprecation notice, migration path, data archival, final audit	Archived Solution record	Migration completion, data retention compliance

23.2 Resource Management

qala tracks all resources consumed across the value chain with full attribution to projects, solutions, teams, and factory tiers:

- Human resources: developer time, team assignments, and capacity planning against roadmap
- Compute resources: CPU, memory, storage, and network per SDE, pipeline run, and deployed instance
- Financial resources: cost allocation per project, team, solution, and environment tier
- Third-party resources: licensed tools, vendor services, and external dependencies with cost attribution
- AI-driven resource optimization: recommends reallocation to reduce waste and improve throughput across the factory

24. Solution Data Platform

The Solution Data Platform manages the full data lifecycle for the qala platform: metrics, events, analytics, master data, and AI-driven insights. Every data point is traceable to its origin Solution, SDE, or Factory. The Data Platform is the observability and intelligence backbone of the entire SFOS.

24.1 Data Platform Capabilities

Capability	Description
Event Streaming	Platform-wide event bus (Apache Kafka) carrying Solution, SDE, Build, Security, AI, and operational events
Metrics Collection	Time-series metrics from all Solutions, SDEs, and Factories: CPU, memory, latency, error rates, build durations
Log Aggregation	Centralized, indexed log collection from all platform services, Solution environments, and CI/CD pipelines
Data Warehouse	Columnar analytics store for historical reporting, trend analysis, and AI model training data
Master Data Management	Authoritative reference data for Solution types, status codes, classification taxonomies, and organizational structures
Analytics Dashboards	Pre-built dashboards for Solution health, factory throughput, defect density, resource utilization, and security posture
AI Inference	Real-time and batch inference endpoints for predictive capabilities throughout the platform
Data Lineage	Full provenance tracking: every data point traces to its origin event, source service, and transformation history
Data Pipeline Orchestration	Managed ETL and ELT pipelines for data ingestion, transformation, and aggregation

24.2 Event Stream Topics

Event Topic	Published By	Key Events Carried
SOLUTION_EVENTS	Solution Registry	Solution creation, update, maturity transitions, retirement
USER_EVENTS	User & Identity Service	User creation, role changes, authentication, access grants/revocations

Event Topic	Published By	Key Events Carried
SDE_EVENTS	SDE Management Service	SDE provisioning, snapshot, rollback, state transitions, quarantine
CMS_EVENTS	Workspace & CMS Service	Content create, update, delete, workspace provisioning
BUILD_EVENTS	Workflow & CI/CD Service	Pipeline runs, build outcomes, stage completions, gate results
ARTIFACT_EVENTS	Artifact & Package Service	Artifact uploads, promotions, downloads, deletions, attestation generation
DATA_EVENTS	Data Platform Service	Pipeline runs, anomaly detections, metric threshold crossings
SECURITY_EVENTS	Security & SEM Service	Threat detections, policy violations, scan results, SDE quarantine events
NOTIFICATIONS	Notifications Service	User and system notifications, alert dispatches, announcement broadcasts
AI_RECOMMENDATIONS	AI Agents Service	Optimization recommendations, predictions, anomaly alerts, self-healing actions

25. AI Integration

AI capabilities are embedded throughout the qala Solution System — not as an add-on layer, but as an integral part of every major subsystem. The AI Agents service provides intelligent hooks, predictions, and automation across all platform functions. AI operates continuously against the Data Platform, consuming metrics, logs, events, and historical patterns to provide real-time and batch intelligence.

25.1 AI Capability Map

AI Capability	Subsystem	Description
SDE Optimization	SDE Management	Recommends configuration improvements for performance, cost efficiency, and tooling selection
Pipeline Bottleneck Detection	CI/CD	Identifies slow pipeline stages and recommends parallelization, caching, or restructuring
Predictive Defect Detection	Testbed	Flags high-risk code areas using historical defect patterns and change impact analysis
Test Case Generation	Testbed	Synthesizes test scenarios from specifications, past failures, and coverage gaps
Self-Healing Tests	Testbed	Automatically updates fragile or stale tests when interfaces, contracts, or APIs evolve
Anomaly Detection	Data Platform	Detects unusual patterns in metrics, logs, security events, and resource utilization
Resource Prediction	Data Platform	Forecasts compute and capacity needs based on historical activity trends and roadmap signals
Security Threat Analysis	Security/SEM	Correlates security events across SDEs and solutions to identify coordinated attack patterns and risks
Prototype Feedback Analysis	Solution Model	Synthesizes prototype results and user feedback into actionable design recommendations
Vendor Advisory Monitoring	Tooling	Monitors the tool and dependency ecosystem for security advisories and recommended upgrades
Backup Schedule Optimization	SDE Management	Recommends backup frequency and retention policies based on change velocity, usage patterns, and risk
Content Suggestions	Workspace/CMS	Recommends documentation structure, template selection, and standards compliance improvements

AI Capability	Subsystem	Description
Template Recommendation	Governance	Suggests the most appropriate SDE template, pipeline definition, or project scaffold for a given context
Value Chain Optimization	Orchestration	Identifies inefficiencies in the end-to-end solution lifecycle and recommends process improvements
AI-Assisted Code Review	CI/CD	Provides code quality, security, and standards compliance suggestions during pull request review

25.2 AI Governance

AI recommendations in qala are advisory by default — they are presented to practitioners and administrators for review and acceptance rather than applied automatically without human oversight. Exceptions include specific self-healing and anomaly response workflows where automated action is explicitly configured and policy-approved.

- All AI recommendations are logged in the audit trail with their basis, confidence score, and disposition (accepted/rejected)
- AI models are retrained on platform telemetry on a configured schedule; model versions are tracked in the Tool Registry
- AI recommendations that are systematically rejected are flagged for model review and recalibration
- Human-in-the-loop controls are enforced for any AI action that modifies CM-maturity environments or production systems

26. Notifications & Communications

The qala Notifications and Communications system provides structured, multi-channel message delivery for all platform events, user actions, and system alerts. Communications are organized into channels, room types, and audience segments to ensure that the right information reaches the right recipients through the right medium.

26.1 Message Types

Message Type	Description	Delivery Pattern
Event Notification	System-generated notification triggered by a platform event (build completion, maturity promotion, SDE quarantine)	Push — delivered immediately on event
Alert	High-urgency notification requiring immediate attention (security threat, critical pipeline failure, SLA breach)	Push + escalation — multi-channel until acknowledged
Announcement	Broadcast communication to a defined audience (platform maintenance, policy update, release notice)	Scheduled or immediate broadcast
Direct Message	Unicast communication between platform users or from a service to a specific user	Direct delivery to recipient
Group Message	Multicast communication to a team, project group, or factory audience	Delivered to all group members
Digest	Aggregated summary of multiple events or notifications delivered on a defined schedule	Scheduled — daily, weekly, or configurable

26.2 Communication Channels

- Email: formal notifications, digests, and compliance-required communications
- In-platform messaging: direct messages, group messages, and room-based collaboration
- Webhook: programmatic delivery to external systems, incident management tools, and automation pipelines
- Phone / SMS: high-priority alerting for critical incidents and on-call escalations
- Voice / Video: room-based collaboration and consultation channels for team coordination

26.3 Collaboration Rooms

qala organizes collaborative contexts into structured rooms that map to specific work types and organizational relationships:

- Work rooms: gigs, consultations, bookings, tasks, jobs, contracts, offers, deals, requests, proposals, bids, orders
- Community and team rooms: organized by team, project, factory, or organizational unit
- Marketplace rooms: exchanges, listings, and offering-discovery spaces
- Cross-factory rooms: enabling collaboration across Solution Factory boundaries within a governed context

27. API & Microservice Architecture

qala is implemented as a collection of independently deployable microservices, each owning its data and exposing a versioned API. Services communicate asynchronously via event streams and synchronously via REST where required. All external and internal API traffic routes through a central API Gateway.

27.1 Service Map

Service	Lang uage	Port	System	Subsystem
qala-kernel	Rust	7000	Kernel	Registry, Health, Command Bus
api-gateway	Go	8080	Control Plane	API ingress + service routing
user-identity-service	Go	8081	Control Plane	Identity and RBAC
sde-management-service	Go	8082	Control Plane	SDE lifecycle, snapshots, solutions, factories
workspace-cms-service	Go	8083	Control Plane	Workspaces and file content
workflow-ci-cd-service	Scala	8084	Execution Plane	Pipeline run and status
artifact-package-service	Go	8085	Control Plane	Artifact metadata and retrieval
data-platform-service	Scala	8086	Execution Plane	Analytics and metrics
ai-agents-service	Rust	8087	Intelligence	Recommendation and analysis
security-sem-service	Rust	8088	Security	Threat, policy, and SDE scanning
notifications-service	Scala	8089	Execution Plane	Message dispatch

27.2 Kernel System

The qala-kernel is the root coordination layer of the entire platform. It maintains system-level state for all subsystems and provides the authoritative source of truth for service health, event telemetry, and subsystem status.

- Registry: tracks registered services and heartbeat status; provides service discovery
- CommandBus: executes kernel commands (set_subsystem_status, set_service_status, register-service)
- EventAggregator: counts and stores recent topic events; provides platform-level telemetry
- Health: exposes system health status across all registered services and subsystems

27.3 Systems & Subsystems

System	Subsystems
Kernel System	Registry, CommandBus, EventAggregator
Control Plane	Gateway, Identity, SDE, Workspace, Artifact
Execution Plane	Workflow (CI/CD), Data (Metrics/Analytics), Notifications
Intelligence & Security Plane	AI (Recommendations/Analysis), SEM (Security Events/Policy/Scanning)

27.4 API Gateway

All external and internal API traffic routes through the central API Gateway, which provides authentication, authorization, rate limiting, caching, routing, and observability for the entire platform API surface.

- Authentication (JWT/OAuth 2.0) and Authorization (RBAC policy enforcement)
- Request routing to the appropriate microservice
- Rate limiting, caching, and request/response transformation
- TLS termination and mutual TLS enforcement for service-to-service calls
- API versioning and backward compatibility management
- Observability: request tracing (OpenTelemetry), latency metrics, and error rate monitoring

27.5 Complete API Surface

Gateway (Go, :8080)

Meth od	Endpoint	Description
GET	/health	Gateway health check
GET	/routes	List all registered routes
ANY	/api/*	Proxy to downstream services

User Identity Service (Go, :8081)

Meth od	Endpoint	Description
POST	/users	Create a new user

Method	Endpoint	Description
GET	/users	List all users
GET	/users/{id}	Get user by ID
PUT	/users/{id}	Update user record
DELETE	/users/{id}	Delete user
POST	/auth/login	Authenticate user, return JWT
POST	/auth/logout	Invalidate session

SDE Management Service (Go, :8082)

Method	Endpoint	Description
POST	/sdes	Create a new SDE
GET	/sdes	List all SDEs
GET	/sdes/{id}	Get SDE by ID
PATCH	/sdes/{id}	Update SDE configuration
DELETE	/sdes/{id}	Delete SDE
POST	/sdes/{id}/snapshot	Capture SDE state snapshot
POST	/sdes/{id}/rollback	Rollback SDE to a prior snapshot
POST	/sdes/{id}/clone	Clone SDE to a new instance
POST	/sdes/{id}/solutions	Associate a solution with this SDE
DELETE	/sdes/{id}/solutions/{solutionId}	Remove solution association from SDE
POST	/solutions	Create a new Solution
GET	/solutions	List all Solutions
GET	/solutions/{id}	Get Solution by ID

Method	Endpoint	Description
PATCH	/solutions/{id}	Update Solution record
DELETE	/solutions/{id}	Delete Solution
POST	/factories	Create a new Solution Factory
GET	/factories	List all Solution Factories
GET	/factories/{id}	Get Factory by ID
PATCH	/factories/{id}	Update Factory
DELETE	/factories/{id}	Delete Factory
POST	/factories/{id}/sdes	Create SDE within a Factory
DELETE	/factories/{id}/sdes/{sdeId}	Remove SDE from Factory

Workspace CMS Service (Go, :8083)

Method	Endpoint	Description
POST	/workspaces	Create a new workspace
GET	/workspaces	List all workspaces
GET	/workspaces/{id}	Get workspace by ID
PUT	/workspaces/{id}	Update workspace
DELETE	/workspaces/{id}	Delete workspace
POST	/workspaces/{id}/content	Upload content to workspace
GET	/workspaces/{id}/content/{filePath}	Retrieve content by file path

Workflow CI/CD Service (Scala, :8084)

Meth od	Endpoint	Description
POST	/pipelines/run	Trigger a CI/CD pipeline run
GET	/pipelines/{id}/status	Get pipeline run status
GET	/pipelines	List all pipeline runs
POST	/pipelines/{id}/cancel	Cancel a running pipeline

Artifact Package Service (Go, :8085)

Meth od	Endpoint	Description
POST	/artifacts/upload	Upload a new artifact
GET	/artifacts	List all artifacts
GET	/artifacts/{id}	Get artifact metadata by ID
GET	/artifacts/{id}/download	Download artifact binary
DELE TE	/artifacts/{id}	Delete artifact record

Data Platform Service (Scala, :8086)

Meth od	Endpoint	Description
POST	/data/pipeline/run	Trigger a data pipeline run
GET	/data/analytics	Retrieve analytics and metrics data
GET	/data/metrics/{key}	Get time-series metrics for a specific key
GET	/data/lineage/{id}	Get data lineage for a specific data record

AI Agents Service (Rust, :8087)

Meth od	Endpoint	Description
GET	/recommendations	Get AI recommendations for the current context

Method	Endpoint	Description
POST	/analyze_sde	Trigger AI analysis of an SDE configuration
POST	/predict/defects	Predict defect risk for a given code change set
POST	/generate/tests	Generate test cases from a specification

Security SEM Service (Rust, :8088)

Method	Endpoint	Description
GET	/threats	List all active security threats
POST	/policy/update	Update a security policy
POST	/scan_sde	Trigger a security scan of an SDE
GET	/vulnerabilities	List vulnerability findings with severity and status
GET	/audit/logs	Retrieve immutable audit log entries

Notifications Service (Scala, :8089)

Method	Endpoint	Description
POST	/notify	Send a notification to a specified recipient or channel
GET	/notifications	List notifications (filtered by recipient, type, or status)
GET	/notifications/{id}	Get notification by ID
PATCH	/notifications/{id}/read	Mark notification as read

Kernel (Rust, :7000)

Method	Endpoint	Description
GET	/health	Kernel health check
GET	/v1/kernel/status	Get full kernel status for all subsystems

Meth od	Endpoint	Description
POST	/v1/kernel/register-servi ce	Register a new service with the kernel
POST	/v1/kernel/events	Publish an event to the kernel event aggregator
GET	/v1/kernel/events	Retrieve recent kernel events by topic and source
POST	/v1/kernel/command	Execute a kernel command (set_subsystem_status, set_service_status)

28. Deployment Architecture

qala is designed for cloud-native deployment with full support for on-premises and hybrid topologies. The platform is containerized throughout, orchestrated via Kubernetes, and configured through declarative infrastructure-as-code. Every deployment configuration is version-controlled and subject to the same CI/CD pipeline governance as application code.

28.1 Infrastructure Stack

Layer	Technology	Purpose
Containerization	Docker	All services and build environments packaged as immutable container images
Orchestration	Kubernetes with Helm charts	Service deployment, scaling, rolling updates, and health management
Service Mesh	Istio or Linkerd	mTLS, traffic management, load balancing, and service observability
Event Streaming	Apache Kafka	Platform-wide event bus; durable, ordered, partitioned event streams
Secret Storage	HashiCorp Vault or cloud-native KMS	Centralized secrets management with dynamic secret generation
Artifact Storage	OCI-compatible registry + S3-compatible object storage	Container images, binaries, packages, and build artifacts
Observability	OpenTelemetry, Prometheus, Grafana, centralized log aggregation	Metrics, traces, logs, and alerting across all services
Infrastructure as Code	Terraform / Pulumi	Declarative, version-controlled infrastructure provisioning
API Gateway	Custom Go gateway (:8080)	Central API ingress, authentication, routing, and rate limiting
Database	PostgreSQL (per-microservice)	Service-owned relational data stores; migration-ready SQL schemas

28.2 Multi-Tenancy & Isolation

- Each Solution Factory operates in a logically isolated Kubernetes namespace within the platform

- Resource quotas are enforced at the factory, team, and SDE level using Kubernetes ResourceQuotas and LimitRanges
- Network policies prevent cross-tenant data access at the infrastructure layer
- Encryption at rest and in transit is mandatory for all tenant data; keys are per-tenant
- Audit logs are immutable, isolated per tenant, and retained per configurable compliance policy

28.3 Scalability

- All microservices are horizontally scalable; no stateful singletons in the critical path
- Auto-scaling policies based on CPU, memory, and custom application metrics (HPA and VPA)
- Build environments are ephemeral and scale to zero when idle; provisioned on-demand per pipeline run
- Event streaming throughput scales with Kafka partition count and consumer group parallelism
- AI inference services scale independently of core platform services
- Database connections managed via connection pooling (PgBouncer or equivalent)

28.4 Persistence Model

qala uses a per-microservice persistence model. Each service owns its data store and exposes access only through its API. Database schemas are defined as migration-ready SQL DDL files under version control.

- Initial implementation: in-memory stores for rapid development iteration + SQL DDL for migration-ready schemas
- Production deployment: separate PostgreSQL database instance per microservice
- Schema files: located under db/postgres/*.sql in the service repository
- Cross-service data consistency: maintained through eventual consistency via the event bus, not shared databases
- Backups: managed by the platform backup service with configurable retention policies and point-in-time recovery

29. Operating Model

The qala operating model describes how the platform functions at runtime — the sequence of operations from an API call through service processing, event propagation, and kernel aggregation. This model applies uniformly across all platform tiers.

29.1 Standard Request Flow

17. Developer or API client calls the API Gateway or a service API directly
18. API Gateway authenticates (JWT/OAuth), authorizes (RBAC), and routes to the appropriate microservice
19. Service processes the request, updates local state in its data store, and returns a response
20. Service emits one or more event envelopes to the Kafka event bus with the SDD topic taxonomy
21. Kernel aggregates event telemetry and updates subsystem health status
22. Analytics, security, notification, and AI services consume events and react asynchronously
23. Notifications service dispatches alerts and notifications to relevant recipients

29.2 Integration Pattern

- HTTP REST APIs for direct synchronous control-plane actions with immediate response requirements
- Event envelopes for asynchronous propagation with the SDD topic taxonomy for loose coupling
- Service-level in-memory repositories for baseline implementation; PostgreSQL for production persistence
- mTLS for all service-to-service communication; JWT for all client-to-service communication

29.3 Observability Model

Every component of the qala platform is instrumented for full observability across three pillars:

- Metrics: time-series performance and business metrics collected via Prometheus, visualized in Grafana
- Traces: distributed request traces collected via OpenTelemetry; exported to Jaeger or equivalent
- Logs: structured, centralized log aggregation from all services and build environments; indexed for search
- Alerts: automated alerting on metric thresholds, error rates, and SLA violations; routed via Notifications service

30. Sustainability, Reusability & Responsible Design

qala is designed with sustainability as a first-class architectural concern. The platform promotes reusability, renewability, and closed-loop solution design across all solution types and factory tiers. Eco-aware solution design principles are embedded in the governance framework and measurable through the platform's analytics capabilities.

30.1 Sustainability Principles

- **Reusability:** solution components, parts, models, and environments are designed for maximum reuse across projects and factory tiers
- **Sustainability:** solutions are built with defined retirement paths and resource-efficient operational profiles
- **Renewability:** solution architectures prefer reversible, modifiable designs over monolithic, locked-in structures
- **Recyclability:** retired solution assets are inventoried, tagged, and made available for reuse in future solutions
- **Closed-Loop Systems:** waste from one solution's lifecycle becomes an input to another; no knowledge or asset is discarded without assessment
- **Eco-Aware Design:** platform energy and power consumption management system tracks resource utilization and identifies optimization opportunities

30.2 Platform Resource Management

qala includes platform-level resource management capabilities that track and optimize energy consumption, network traffic, and compute utilization across the entire factory hierarchy:

- **Platform Energy / Power Consumption Management System:** tracks energy usage per SDE, pipeline run, and deployed solution; recommends optimization
- **Platform Network Traffic Management:** monitors and governs data transfer volumes; identifies unnecessary cross-service communication
- **Resource Waste Detection:** AI-driven identification of idle SDEs, unused artifacts, and oversized compute allocations
- **Sustainability Dashboards:** reporting on resource efficiency trends, carbon footprint proxies, and optimization opportunities

Appendix A — Glossary

Term	Definition
Solution	The central root element of qala: a typed, versioned answer to a problem or goal, composed of Components and governed by a lifecycle
SDE	Solution Development Environment — the atomic operational unit providing all tools, configuration, and resources for creating and managing a Solution
SF	Solution Factory — a coordinated collection of networked SDEs producing Solutions in a consistent, repeatable way
SFOS	Solution Factory Operating System — the qala platform itself
Solution Book	The complete knowledge and documentation repository for a Solution: charter, notes, schedules, WBSs, playbooks, and all reference content
Solution Package	A bundled, versioned, distributable unit containing a Solution artifact, its manifests, dependencies, and deployment instructions
Solution Chain	An ordered, dependency-linked sequence of Solutions forming an end-to-end value chain
Solution Set	An unordered collection of Solutions deployed or managed together
Solution Kit	A curated, pre-configured bundle of Solutions, tools, and configurations targeting a specific class of problem
Tool Solution	A Solution classified as a tool, consumed by other Solutions or SDEs as a tooling dependency
Solution Portfolio	A governed collection of Solutions managed under a common Factory or organizational unit
CM	Configuration Management — governed control of all environment and system configuration; the highest maturity stage
CI	Continuous Integration — automated build and test on every code change
CD	Continuous Deployment/Delivery — automated promotion of validated artifacts to target environments
SAST	Static Application Security Testing — code analysis without execution
DAST	Dynamic Application Security Testing — testing against a running application
IAST	Interactive Application Security Testing — instrumented testing during execution
SCA	Software Composition Analysis — dependency and supply-chain vulnerability scanning

Term	Definition
RBAC	Role-Based Access Control — access permissions determined by assigned roles
SLSA	Supply chain Levels for Software Artifacts — a security framework for build attestation and provenance
Toolchain	A linked, ordered sequence of Tools where each tool's output feeds the next
Blueprint	The formal structural specification of a Solution Model
Maturity	The lifecycle classification of a Solution: SANDBOX, DEV, NIGHTLY, TEST, or CM
Hermetic Build	A fully isolated, reproducible build environment with all dependencies frozen and no external access
Value Chain	The end-to-end sequence of activities transforming a Solution concept into an operational Solution
WBS	Work Breakdown Structure — hierarchical decomposition of all work required to deliver a Solution
MDM	Master Data Management — ensuring consistency of shared reference data across platform services
MTTR	Mean Time to Resolution — average time from defect or incident discovery to resolution
MTTD	Mean Time to Detect — average time from a defect or incident occurrence to its detection
PII	Personally Identifiable Information — data that can identify a natural person
IaC	Infrastructure as Code — infrastructure provisioning defined and managed as version-controlled code
POC	Proof of Concept — a prototype used to validate feasibility of an approach
CPQ	Configure-Price-Quote — a structured system for modeling solution offerings, pricing, and customer configurations
DAG	Directed Acyclic Graph — the data structure used to model workflow task dependencies in qala orchestration
SEM	Security and Event Management — the qala security coordination service for threat detection, policy enforcement, and SDE scanning

Appendix B — Solution Maturity Gate Criteria

Maturity Promotion	Required Gate Criteria
SANDBOX → DEV	Solution record created in Registry; SDE provisioned; primary owner assigned; value proposition documented; Solution type declared
DEV → NIGHTLY	CI pipeline configured and passing; build succeeds on main branch; unit tests present and passing; code quality baseline established; no critical security findings from SAST
NIGHTLY → TEST	All unit tests passing; all integration tests passing on main branch; SAST scan clean; SCA scan clean (no known critical CVEs); no P1 defects open; performance baseline captured and documented
TEST → CM	All test suites (unit, integration, system, performance, security) passing; DAST clean; container scan clean; penetration test completed with no unmitigated P1 findings; performance benchmarks met; governance board review completed; CM board formal sign-off; build attestation (SLSA) generated for release artifact; release changelog complete; deployment runbook reviewed

Appendix C — Database Schema Overview

qala uses a per-microservice PostgreSQL database model. The following tables represent the core data structures for each service. Schema files are located at db/postgres/*.sql in each service repository.

Service	Core Tables
User Identity	Users(id PK, name, email UNIQUE, role, created_at), Sessions(id PK, user_id FK, token, expires_at)
SDE Management	SDE(id PK, owner_id FK, template_id FK, status, snapshot_ver INT, maturity), SDE_Snapshots(id PK, sde_id FK, version INT, timestamp, storage_location)
Solutions	Solutions(id PK, name, type, version, maturity, owner_id FK, sde_id FK, factory_id FK, created_at, updated_at), Solution_Features(id PK, solution_id FK, name, brief, feature, status)
Solution Factories	Factories(id PK, name, tier, owner_id FK, parent_factory_id FK), Factory_SDE(factory_id FK, sde_id FK)
Workspace CMS	Workspaces(id PK, name, owner_id FK, type, created_at), Files(id PK, workspace_id FK, path, content BLOB, version, created_at)
Workflow CI/CD	Pipelines(id PK, name, solution_id FK, status, triggered_by FK, created_at), Pipeline_Stages(id PK, pipeline_id FK, name, status, duration_ms), Pipeline_Logs(id PK, pipeline_id FK, stage_id FK, message, timestamp)
Artifacts	Artifacts(id PK, name, version, solution_id FK, sde_id FK, attestation_hash, storage_uri, created_at), Artifact_Attestations(id PK, artifact_id FK, build_id, commit_hash, env_hash, signature, slsa_level)
Data Platform	Metrics(id PK, source, key, value, timestamp), Logs(id PK, source, level, message, timestamp), MasterData(id PK, type, key, value, last_updated)
Security SEM	Threats(id PK, type, severity, detected_at, sde_id FK, status), Policies(id PK, name, rules JSONB, last_updated), Vulnerabilities(id PK, cve_id, severity, source, remediation_status, detected_at)
Notifications	Notifications(id PK, target_id FK, channel, type, message, status, created_at, read_at)

Appendix D — SDD Traceability Matrix

This matrix maps capabilities in this design document to concrete implementation locations in the qala repository.

Capability	Implemented In
API Gateway routing (/api/*)	go/cmd/api-gateway/main.go
User CRUD + USER_EVENTS	go/cmd/user-identity-service/main.go
SDE lifecycle + snapshot/rollback + SDE_EVENTS	go/cmd/sde-management-service/main.go
Solution hierarchy model + solution factory orchestration	go/cmd/sde-management-service/main.go
Workspace CRUD + content operations + CMS_EVENTS	go/cmd/workspace-cms-service/main.go
Pipeline run/status + BUILD_EVENTS	scala/workflow-ci-cd-service/src/main/scala/com/qala/workflow/Main.scala
Artifact upload/download + ARTIFACT_EVENTS	go/cmd/artifact-package-service/main.go
Data pipeline + analytics + DATA_EVENTS	scala/data-platform-service/src/main/scala/com/qala/data/Main.scala
AI recommendations + SDE analysis + AI_RECOMMENDATIONS	rust/ai-agents-service/src/main.rs
Security threats/policies/scan + SECURITY_EVENTS	rust/security-sem-service/src/main.rs
Notification dispatch + NOTIFICATIONS	scala/notifications-service/src/main/scala/com/qala/notifications/Main.scala
Kernel OS coordination (systems/subsystems/events/commands)	rust/qala-kernel/src/main.rs, rust/qala-kernel/src/state.rs
Shared event contract	contracts/events/schema.json, rust/qala-shared/src/lib.rs, go/internal/platform/events/events.go
DB schema blueprint	db/postgres/*.sql